

面向城市空间任务智能处理的规划智能体：研究方案

【注：标题暂定为这个表述，可以修改】

一、研究问题

总问题：如何构建一个面向城市时空数据分析多智能体系统的任务规划层，功能上使其能够生成结构化、可验证、可迭代优化的执行方案，从而有效解决当前规划层存在的任务分解不合理、依赖关系丢失和错误、误差传播等问题，准确处理自然资源、城市规划、可持续性等综合性强耦合度高的任务？

问题类型：任务规划与分解缺陷（C类，主）和知识库覆盖不足（A类，副）。

这个问题可以分为以下三个问题：

- RQ1** 如何构建面向任务规划及行业领域的知识表示框架，统一存储GIS工具的功能特征、参数和环境依赖和兼容性约束？
- RQ2** 如何设计基于有向无环图的显式任务依赖建模方法，将用户需求自动转换为包含数据流依赖和顺序约束的、可直接验证的结构化执行计划？
- RQ3** 如何设计规划和执行的闭环验证与迭代优化机制，根据执行阶段的反馈自动修正规划缺陷？

二、论点及假设

多智能体架构是实现综合性强耦合度高的时空分析任务执行的唯一可行架构。

- 将公认复杂任务从通用时空任务知识中独立出来（自然资源管理和分析、可持续性分析、城市规划、智慧城市等），构建领域专门化的**知识表示框架**，能够显著提升任务分解的合理性和完整性；目前所谓复杂时空分析任务能够拆解为以下两个类别：
 - 模式明确，操作复杂：如服务区、生活圈、设施选址、密度、空间统计；
 - 结论开放，操作复杂：如相关性、异质性、相似性、挖掘。基于监督分类的知识表示框架即可进行构建。
- 将规划的输出形式从自然语言描述转为**基于有向无环图的工作流结构**，配合显式的依赖约束（数据流依赖和工具兼容性约束）能够有效减少执行阶段的依赖错误和顺序错误。空间任务执行方案有固定的受众人群和执行对象，其前提是空间分析建模，具体表现为分析任务的结构化表示。但目前通用大模型不可能独立完成空间分析建模的结构化，必须借助外部知识；
- 主动引入规划——执行的**闭环反馈机制**，响应智能体的执行失败信息，从而反向修正规划，显著提升复杂任务1~2次执行成功率。

三、创新性参考

- 提出面向任务规划和行业领域的有向无环图显式依赖建模方法，解决现有规划层组分的输出不可计算、依赖关系丢失的问题。
- 设计规划与执行闭环迭代机制，首次将执行失败信息结构化反馈给规划层，实现规划自修正。
- 构建GIS任务规划知识增强体系，即任务模式库和工具依赖约束表，提升了规划层对领域知识的利用。
- 在实际地理时空数据环境（PostGIS+GPKG+轨迹数据）中验证了方法的有效性，相比原有线性规划+后校验方案，复杂任务成功率提升X个百分点。

四、成文大纲

- 绪论（内容提示）

时空分析智能体是推动地理信息科学向空间智能计算处理转化的基础设施和必由之路。当前地理时空分析智能体在一定程度上降低了空间分析上手的复杂性，解放了为重复工作流程和繁杂语义规划所束缚的生产力，但能够做的任务多为单元测试和线性流程，这与实际自然资源、城市规划、可持续发展、智慧城市等任务的语义限定极为开放的特征，联合导致智能体应用窄推广难的难题，遑论其在实践中将单元测试也当作分析任务看待。

1. 在软件架构和任务流程方面，从原型开始，经历线性工作流规划、单智能体架构，直到现如今多智能体架构，系统完整性得到了长足的发展，智能体已有分工协作；

2. 工具方面，从有限工具到接入开源软件完整生态，工具体系从窄和小已发展到了大而全，再发展的空间逐渐收窄；

3. 任务规划方面，早期要求一次性给出处理计划（一步到位），规划错误信息反馈到专门模块修正（转嫁矛盾）。随着任务耦合性和复杂性的增强这个中间流程才分离出来，但目前本模块的存在性未有定论（可有可无）且职责也不清晰（可大可小），这就归结于空间分析建模即结构化表示的作用，建模尺度是锦上添花。

经过近年的发展，规划的问题在工具、记忆、计划和执行四要素中被日益凸显出来，缺少规划层会导致如终止判断不准确、步骤错误累积、子任务分解缺失或不合理的运行不稳定问题，影响自动空间分析准确性甚至于性能，日益成为目前制约智能执行的主要矛盾。

【之后参考上面的】

2. 国内外研究现状（内容提示）

- 2.1 GeoAI的产生：深度学习与地理信息科学结合
- 2.2 从GeoAI到LLM4GIS：大模型对地理信息科学的影响
- 2.3 Autonomous GIS：当地理空间分析自动化之后（可选）

3. 研究方法和技术路线（内容摘要）

3.1 总体设计

层级	功能	对应问题
L1: 规划知识层	构建工具与任务模式的知识图谱，存储工具功能特征、参数类型和数据依赖	RQ1 (知识)
L2: 任务规划层	将用户自然语言需求解析为基于有向无环图的结构化执行计划	RQ2 (规划)
L3: 执行代理层	根据结构化规划调用具体平台工具，捕获执行结果和错误信息	RQ3 (验证)
L4: 规划迭代优化层	分析执行失败原因，定位规划缺陷，自动修正规划方案	RQ3 (验证)

3.2 各层概述

3.2.1 规划知识层——面向任务规划和行业领域的地理空间知识图谱构建

现有GeoCode-Base等知识库主要服务于代码生成或工具调用阶段的知识检索，缺少专门服务于**任务规划**的知识结构。规划阶段的核心要求不是具体的工具实现或代码语法（比如函数用法，其参数的类型和约束），而是任务的**逻辑分解范式**（如选址问题是缓冲区、相交、联合、擦除的标准操作链）。

技术路线：

1. 知识类型设计。将规划知识分为两个层次：

- **任务模式层**：从已有问题中提取**标准解决范式**（如设施选址是Buffer/Intersect/Union/Erase、可达性分析是Buffer/Intersect/Summary Statistics的模式），通过专家标注或LLM辅助归纳的方式构建初始模式库。
- **工具能力描述层**：每个工具抽象为“输入类型→操作类型→输出类型”的函数签名（如Buffer: (几何类型|点/线/面, 距离数值) → (几何类型|面)）。该签名不绑定具体平台，通过抽象出的表述方式（如Buffer）到平台实现（PyQGIS的native:buffer/ArcPy的Buffer_analysis）的映射关系实现平台无关性。

2. 存储结构：采用图数据库（如Neo4j）或JSON/YAML文本存储，**任务模式节点**作为规划的高层指导，工具能力描述节点作为规划的原子单元，帮助规划层在任务模式无法完全匹配时实现**泛化**——通过查找工具能力描述库，构建与目标任务逻辑等价但工具组合不同的替代方案。

3.2.2 任务规划层——基于有向无环图的显式依赖建模

当前GeoAgent等框架的规划输出仅为Markdown文档，子任务间的依赖关系（数据依赖、顺序约束）未被显式表达，导致执行阶段出现顺序混乱或数据丢失。核心难点在于让LLM将自然语言需求准确转换为图结构，同时保证图的无环性和依赖的完整性。

技术路线：

1. 规划输出结构设计：以有向无环图为核心数据结构，优点是若将图改为无向图即可追踪 workflow，增强可解释性。

- **节点**：子任务，包含字段：**task_id**、**description**、**tool_needed**（所需工具）、**input_schema**（输入所需数据类型）、**output_schema**（输出数据类型）。
- **有向边**：子任务间的数据依赖关系和数据流向（如**data_source**、**data_target**）。

2. 规划生成方法：两阶段约束生成策略，以提示词为主。

- **第一阶段（粗粒度）**：LLM根据L1的规划知识库，将用户需求分解为子任务序列（如“①加载数据→②创建缓冲区→③相交分析”）。
- **第二阶段（细粒度）**：LLM为每个子任务声明输入输出类型，并识别跨子任务的数据依赖关系，构建DAG。
- 在第二阶段的提示中，嵌入若干约束规则作为硬性约束，如所有输入必须有至少一个对应的上游节点提供、DAG不可存在循环依赖等。

3. DAG结构定义示例：

```
{
  "plan_id": "plan_20260603_001",
  "nodes": [
    {
      "task_id": "task_1",
      "description": "加载福田区道路数据",
      "tool_primitive": "load_vector",
      "input_schema": {"data_source": "postgresql", "layer": "roads_futian"},
      "output_schema": {"type": "GeoDataFrame", "crs": "EPSG:4326", "geometry":
"LineString"}},
      "reflect_checks": ["crs_defined", "non_empty"]
    },
    {
```

```

    "task_id": "task_2",
    "description": "对学校点创建500米缓冲区",
    "tool_primitive": "buffer",
    "input_schema": {"type": "GeoDataFrame", "geometry": "Point"},
    "output_schema": {"type": "GeoDataFrame", "geometry": "Polygon", "crs":
"EPSG:3857"},
    "parameters": {"distance": 500, "unit": "meters"}
  }
],
"edges": [
  {"from": "task_1", "to": "task_2", "data_flow": "roads_layer -> buffer_input"},
  {"from": "task_2", "to": "task_3", "data_flow": "buffer_output ->
intersect_input"}
],
"global_constraints": [
  "所有空间操作的CRS必须统一为EPSG:3857",
  "缓冲区距离单位必须为米"
]
}

```

4. 验证方式：编写DAG验证器（基于networkx），自动检查无环性、输入输出类型匹配、约束满足性。

3.2.3 执行代理层和规划迭代优化层——规划与执行的闭环验证

当前GIS多智能体系统的错误传播问题主要源于规划层的错误在执行阶段被放大且未被反馈修正。核心难点在于：需要将执行阶段的原始错误信息（如“参数类型不匹配”“数据格式不兼容”）映射回规划层的具体缺陷类型（如“工具选择错误”“依赖约束缺失”），这对错误诊断的知识库要求较高。

技术路线：

1. 执行跟踪与错误捕获：在执行代理层执行每个子任务时，记录如下状态：
 1. 任务异常信息：如工具不存在ToolNotFoundError、参数类型不匹配TypeError、输入缺失InputError、依赖顺序错误SequenceError；
 2. 任务状态：状态只有成功和失败之分；
 3. 关键数据状态：数据大小、参考系等。
2. 失败原因诊断与规划修正：设计独立的规划审核代理（Plan Auditor Agent），其工作流程为：
 - 一诊断：审核代理接收执行日志和原始规划DAG，分析失败原因并判断其根源是否在于规划层，即基于预设的诊断规则库判断失败根源。例如参数类型不匹配对应L2阶段工具抽象原语与实际平台实现不匹配；缺失上游数据对应L2阶段依赖关系提取不完整。
 - 二定位：然后在原始规划DAG中标识出具体的问题节点或边。
 - 三修正：审核代理生成修正指令JSON，触发规划层重新生成修改后的DAG，替代原规划继续执行。

```

{
  "diagnosis": "CRS mismatch between task_1 output (EPSG:4326) and task_2
requirement (EPSG:3857)",
  "fault_location": "edge between task_1 and task_2",
  "suggested_fix": "Insert a new task 'reproject' between task_1 and
task_2, setting target_crs=EPSG:3857",
  "new_plan_constraints": ["All layers must be in EPSG:3857 before

```

```
    spatial operations"]
  }
```

3. **修正知识积累**：将每次修正成功的“规划缺陷→修正方案”案例对存储到专门的知识库中，作为未来规划的参考（few-shot示例）。该积累机制旨在逐步降低修正的token消耗和延迟，使系统在运行中持续优化。

4. 实验设计（基于TerraX现有数据环境，内容摘要）

4.1 数据条件

利用TerraX已有的数据环境：

数据源	时效	区域	可构建任务类型示例
OSM	2026.06	大湾区（除肇庆和江门）	设施选址、交通可达性分析、土地利用叠置
轨迹	2022.07	深圳	热点区域识别、轨迹密度分析、OD流统计
高德	2024	大湾区（除肇庆和江门）	空间查询、核密度分析
水系	2022		水系缓冲区、叠置分析

4.2 测试任务集创建

从这些数据中人工构造**90个空间分析任务**，分为****涉及工具数量 T 和空间关系 R ****两个维度，空间关系如下表：

R等级	描述	示例
R1	单一图层、单个原子操作	计算几何、按属性/值/位置筛选、IDW插值
R2	两个图层、基础空间关系（距离/包含/相交）	点-面包含、线-面相交、缓冲区
R3	多个图层、多步骤逻辑（与/或/非组合）	(A near B) AND (A not near C)、等时圈网络、设施分布
R4	多源异构数据、时空联合、统计推断	轨迹密度与POI相关性分析、选址

然后根据下表要求构造任务。**实际格式和内容要求参见实验方案的1.2和1.3部分。**

T\R	R1	R2	R3	R4	小计
T1-3	12	9	0	0	21
T4-7	9	15	6	0	30
T8-12	0	6	18	3	27
T12+	0	0	3	9	12
小计	21	30	27	12	90

4.3 评估指标摘要

指标	定义	对比对象
规划合理性	生成的DAG无环+类型匹配+约束满足的比例	对比旧版线性Plan vs 新版DAG
首次执行成功率	无需修正直接成功的任务比例	对比无迭代机制 vs 有迭代机制
最终成功率	经过≤3次迭代修正后成功的任务比例	评估闭环迭代的增益
平均修正次数	成功任务所需的平均迭代次数	评估诊断效率
诊断准确率	审核代理定位的错误根因与人工判断一致的比率	人工抽检

4.4 对比基线（消融实验）

基线	描述	来源
Baseline 1: TerraX原版	线性Plan + 执行后校验（无闭环修正）	已有实现
Baseline 2: GeoAgent（仅执行层）	参考Lin et al.的无规划层版本	论文数据
Baseline 3: GPT-4o直接生成代码	无规划，直接输出可执行脚本	单Agent对比
Our Method: DAG Planner + Plan Auditor	本方案	—

4.5 预期结果

任务复杂度	TerraX原版成功率	本方案预期成功率	提升幅度
简单(1-2工具)	~90%	~95%	+5%
中等(3-4工具)	~70%	~85%	+15%
复杂(5+工具)	~50%	~75%	+25%

特别地，对于因**CRS不匹配、依赖顺序错误、输入输出类型不匹配**导致的失败，本方案的闭环迭代修正应能恢复其中60-80%。

5. 结论与讨论

五、实验方案

1. 资源

1.1 空间数据及数据库

参见成文大纲的4.1部分

1.2 空间任务模式库

任务模式库是时空分析建模的抽象化结果集合，由如下三个特征定义：

- 1. 某一类任务（如设施选址、可达性分析、热点探测）的**标准操作链**，操作链以抽象工具原语表示；
- 2. 操作链中的**数据流依赖关系**，即哪个工具的哪个输出作为哪个工具的输入；
- 3. 操作链的**约束条件**，如CRS要求、几何类型要求、参数取值范围。

一个模式统一视为对应一个结构，表现为JSON Schema，如下所示。

```
{
  "pattern_id": "PATTERN_006",
  "pattern_name": "设施选址（多约束）",
  "pattern_name_en": "Facility Siting with Multiple Constraints",
  "description": "给定候选区域或全区域，根据多个空间约束条件（如靠近A、远离B、位于C类用地内）筛选出符合条件的区域",
  "applicable_scenarios": ["学校选址", "垃圾填埋场选址", "商业网点布局"],
  "required_capabilities": ["缓冲区", "相交", "擦除", "属性筛选"],
  "standard_workflow": { // 标准操作链
    "nodes": [
      {
        "task_id": "T1",
        "tool_primitive": "buffer",
        "description": "对靠近类要素创建缓冲区",
        "input_schema": {"type": "GeoDataFrame", "geometry": "Point|Polygon"},
        "output_schema": {"type": "GeoDataFrame", "geometry": "Polygon"},
        "parameters": {"distance": "{{distance_A}}", "unit": "meters"}
      },
      {
        "task_id": "T2",
        "tool_primitive": "buffer",
        "description": "对远离类要素创建缓冲区",
        "input_schema": {"type": "GeoDataFrame", "geometry": "Point|Polygon"},
        "output_schema": {"type": "GeoDataFrame", "geometry": "Polygon"},
        "parameters": {"distance": "{{distance_B}}", "unit": "meters"}
      },
      {
        "task_id": "T3",
        "tool_primitive": "intersect",
        "description": "相交靠近类缓冲区与候选区域",
        "input_schema": [{"ref": "T1.output"}, {"ref": "study_area"}],
        "output_schema": {"type": "GeoDataFrame", "geometry": "Polygon"}
      },
      {
        "task_id": "T4",
        "tool_primitive": "erase",
        "description": "擦除远离类缓冲区",
        "input_schema": [{"ref": "T3.output"}, {"ref": "T2.output"}],
        "output_schema": {"type": "GeoDataFrame", "geometry": "Polygon"}
      },
      {
        "task_id": "T5",
        "tool_primitive": "attribute_filter",
        "description": "按用地类型筛选",
        "input_schema": {"ref": "T4.output"},
        "output_schema": {"type": "GeoDataFrame"},
        "parameters": {"field": "landuse_type", "operator": "in", "value": "{{allowed_landuse}}"}
      }
    ],
    "edges": [ // 数据流依赖约束
      {"from": "T1", "to": "T3"},

```

```

    {"from": "T2", "to": "T4"},
    {"from": "T3", "to": "T4"},
    {"from": "T4", "to": "T5"}
  ],
  "global_constraints": [ // 全局约束
    "所有输入图层必须统一到投影坐标系（如EPSG:3857）后进行缓冲区分析",
    "缓冲区距离单位为米"
  ]
},
"parameters": { // 可配置参数
  "distance_A": {"type": "number", "unit": "meters", "description": "靠近距离阈值"},
  "distance_B": {"type": "number", "unit": "meters", "description": "远离距离阈值"},
  "allowed_landuse": {"type": "list", "description": "允许的用地类型列表"}
},
"input_slots": { // 定义任务所需输入数据角色
  "study_area": {"type": "GeoDataFrame", "description": "研究区域（可选，如果不提供则默认为全区域）"},
  "near_features": {"type": "GeoDataFrame", "description": "需要靠近的要素图层"},
  "avoid_features": {"type": "GeoDataFrame", "description": "需要远离的要素图层"},
  "landuse_layer": {"type": "GeoDataFrame", "description": "用地类型图层"}
}
}

```

1.3 空间任务列表库

一个测试任务对应一个模式，选择该模式之后，具体数据源和参数值在技术上即为填空。模式库内部的`{{allowed_landuse}}`、`{{distance_A}}`等这种用两个大括号括起来的内容即为填空位置。首先需要明确定义，如下JSON所示。

```

{
  "task_id": "TASK_015",
  "natural_language": "在福田区内，筛选出同时满足以下条件的区域：距离学校500米以内、距离工厂1000米以外、且位于商业用地上。",
  "pattern_id": "PATTERN_001",
  "instantiations": {
    "study_area": {"data_source": "futian.gpkg", "layer": "district_boundary"},
    "near_features": {"data_source": "futian.gpkg", "layer": "schools"},
    "avoid_features": {"data_source": "futian.gpkg", "layer": "factories"},
    "landuse_layer": {"data_source": "futian.gpkg", "layer": "landuse"},
    "distance_A": 500,
    "distance_B": 1000,
    "allowed_landuse": ["商业用地", "商住混合用地"]
  },
  "expected_plan_dag": {
    // 由任务模式库自动生成，存储为参考答案
  },
  "complexity": {"tools": 5, "spatial_cognition": "R3"}
}

```


对于expected_plan_dag属性，一般通过三步生成：

- 1. **workflow加载** : 加载对应任务模式PATTERN_006的standard_workflow；
- 2. **数据源填写** : 将本任务列表instantiations中的具体数据源替换模板中的占位符，如{{study_area}}填写为futian.gpkg: district_boundary；
- 3. **参数值填写** : 将本任务列表instantiations中的具体参数值填至模板中的占位符，如{{distance_A}}填写为500；
- 4. **有向图输出** : 如下JSON所示，填入上面的JSON。

```
{
  "nodes": [
    {"task_id": "T1", "tool_primitive": "buffer", "input": "futian.gpkg: schools",
"distance": 500, ...},
    {"task_id": "T2", "tool_primitive": "buffer", "input": "futian.gpkg: factories",
"distance": 1000, ...},
    ...
  ],
  "edges": [{"T1", "T3"}, {"T2", "T4"}, ...]
}
```

1.4 空间分析工具库

沿用目前智能体的工具体系。

1.5 运行异常和错误知识库

目前使用如下15种异常和错误类型。

ID	error_pattern 异常和错误关键词	error_category 异常和错误大类	possible_planning_defects 原因枚举	suggested_fix 建议修复措施	confidence 置信度
E01	CRS, coordinate reference system, projection, reproject, cannot transform, incompatible CRS	坐标参考系统	规划中未包含CRS统一或重 投影步骤；或各图层CRS不 一致但未处理	在所有空间操作之前 插入一个reproject 任务，将所有输入图 层转换为统一的目标 CRS（如EPSG:3857）	高
E02	geometry type, expected Polygon but got Point, LineString	几何类型不匹 配	上游工具输出的几何类型与 下游工具要求的输入类型不 一致	在上游工具后插入一 个几何类型转换任务 （如centroid: 面→ 点；boundary: 面→ 线；buffer(0): 点/ 线→面）	高

ID	error_pattern 异常和错误关键词	error_category 异常和错误大类	possible_planning_defects 原因枚举	suggested_fix 建议修复措施	confidence 置信度
	cannot be, geometry mismatch				
E03	field not found, KeyError, 'xxx' is not a column, no attribute named	字段不存在	规划中引用的字段名与数据源实际字段名不一致（可能因大小写、别名、截断等原因）	使用数据理解工具重新读取目标图层的实际字段列表，修正规划中的字段引用	高
E04	file not found, No such file or directory, cannot locate	数据源缺失	规划中引用的数据源路径错误，或依赖的上游任务未成功生成输出文件	检查规划中所有 <code>input_schema</code> 引用的数据源是否已在 <code>data_registry.json</code> 中注册；检查依赖边是否正确	高
E05	empty, no features, no data, Geometry is empty, zero results	输出为空	规划中的空间操作（如缓冲区、相交）参数不当，导致结果为空；或约束过于严格	放宽约束参数（如扩大缓冲区距离、放宽筛选条件），或检查上游数据是否确实包含符合条件的要素	中
E06	memory, out of memory, Killed, swap	资源不足	一次性处理数据量过大；或规划中未对大数据集进行采样/分块	在数据加载后插入一个采样任务（如随机10%），或将大数据操作拆分为分块处理	中
E07	null value, NaN, None, missing value	空值异常	规划中未对输入数据中的空值字段进行清洗或填充	在读取数据后、进行分析前插入一个数据清洗任务（如删除空值行或填充默认值）	中
E08	type mismatch, float, string, cannot convert, expected number	参数类型错误	规划中参数值的数据类型（如字符串“500”而非数字500）与工具要求的类型不符	在调用工具前插入一个参数类型转换（由LLM根据工具签名自动修正）	高
E09	attribute error, has no	API调用错误	规划中调用的工具名称或方法名与	核对工具名称，修正为注册表中的标准名称	高

ID	error_pattern 异常和错误关键词	error_category 异常和错误大类	possible_planning_defects 原因枚举	suggested_fix 建议修复措施	confidence 置信度
	method, not callable		tool_registry.json中注册的名称不一致		
E10	permission denied, read-only, locked	文件权限	输出路径不可写，或文件被其他进程占用	更换输出路径（如使用临时目录），或提示用户关闭占用文件的程序	低
E11	timeout, took too long, exceeded	执行超时	规划中的某些操作（如大规模缓冲区）计算量过大	插入一个数据简化步骤（如简化几何、缩小研究范围），或分批次执行	中
E12	index out of bounds, list index out of range	索引越界	规划中假设某个列表/数组有足够元素，但实际不足（如轨迹分析中时间段超出范围）	增加对数据量/索引范围的预检查，或在参数范围不足时采用默认值	中
E13	duplicate, primary key violation, already exists	重复冲突	规划中尝试写入已存在的表或字段，且未设置覆盖/追加模式	在写入操作前增加去重或检查是否存在，并选择覆盖模式	低
E14	invalid geometry, self-intersection, ring not closed	几何无效	输入数据本身存在几何错误（如自相交多边形），规划中未对其进行修复	在空间操作前插入一个几何修复任务（如make_valid）	中
E15	missing dependency, undefined variable, name '.*' is not defined	缺失依赖	规划中某个任务的输出被下游引用，但该输出在DAG中未定义或未生成	检查DAG的边定义，确保所有被引用的输出都有对应的上游任务节点	高

2. 实验设计

2.1 完整架构及评价指标

分为过程指标、结果指标和全局指标，一共14个。

2.1.1 过程指标

指标	计算方法	注意事项
节点匹配度	工具语义对齐结果匹配度（如buffer vs buffer）	允许功能等价但顺序不同
边匹配度	数据流依赖是否一致	允许添加 必要中间节点 ，如自动插入reproject
参数匹配度	关键参数（距离、字段）是否正确	允许数值近似误差，如500±1m。不得超过1.5m
约束完备性	是否满足全局约束（如CRS统一）	必须完全满足

2.1.2 结果指标

在实际专家评估中，模型规划准确性由多方因素决定，因此采用以下指标：

类别	性质	定义	学名	示例
完全正确	正确	节点、边、参数、约束全部正确	正确率	与参考答案完全一致或功能等价
功能正确但顺序不同	正确	工具正确，但无关依赖的步骤顺序不同	正确率【同上】	先建立学校缓冲再建立工厂缓冲（两者无依赖）
缺少必要步骤	错误	缺少参考答案中的某个节点	缺省率	忘记CRS统一步骤
多余步骤	错误	添加了不必要的节点	冗余率	做了两次同质缓冲区
参数错误	错误	工具正确但参数值错误	参数一致率	距离500写成50
工具选择错误	错误	使用了不合适的工具	工具一致率	用Clip代替Erase
完全错误	错误	DAG结构与参考答案无相似性	错误率	—

2.1.3 全局指标

指标	定义	解释
首次成功率	无需修正直接成功的任务比例	对比无迭代机制 vs 有迭代机制
最终成功率	经过≤3次迭代修正后成功的任务比例	评估闭环迭代的增益
平均执行次数	成功任务所需的平均迭代次数	评估诊断效率
诊断准确率	审核代理定位的错误根因与人工判断一致的比率	人工抽检

2.2 消融实验

基线	描述	来源
Baseline 1: 目前智能体原版	线性Plan + 执行后校验（无闭环修正）	已有实现
Baseline 2: GeoAgent（仅执行层）	参考Lin et al.的无规划层版本	论文数据
Baseline 3: GPT-4o直接生成代码	无规划，直接输出可执行脚本	单Agent对比
Our Method: DAG Planner + Plan Auditor	本方案	—

指标方面有可能需要执行时长对比

3. 实验日志

```
{
  "task_id": "T3_S2_001",
  "task_description": "...",
  "method": "terrax_original | terrax_dag | terrax_dag_auditor",
  "iteration": 0, // 第几次修正迭代
  "success": true/false,
  "final_success": true/false, // 经过max_iter后是否成功
  "plan_dag": {...},
  "execution_log": [...],
  "auditor_diagnosis": {...} (if failed),
  "time_seconds": 12.5,
  "token_usage": {"prompt": 1200, "completion": 450}
}
```

记录如上即可。